

Real Time Information Systems

Written by Nathan KIEFFER

Schedule Search For a Real Time System

SOMMAIRE

Introduction	1
I SYSTEM MODELING AND WCET EXTRACTION	3
I.1 WCET Evaluation for τ_1	3
I.2 Data Scaling Strategy	3
II SCHEDULABILITY AND OPTIMIZATION	4
II.1 Processor Utilization and Hyperperiod	4
II.2 Algorithmic Complexity	4
II.3 Minimizing Wait Time and Maximizing Idle Time	4
III SCHEDULING SCENARIOS AND RESULTS	5
III.1 Scenario 1: Strict Schedule (No Deadlines Missed)	5
III.2 Scenario 2: Relaxed Constraints (τ_5 Delay)	6
CONCLUSION	7



Introduction

This report details the exhaustive schedulability analysis and non-preemptive schedule formulation for a specific periodic task set. The core objective is to map out a deterministic execution flow where no temporal constraints are violated, whilst minimizing the cumulative waiting time of the spawned jobs.

Furthermore, this analysis investigates the system's flexibility by testing a relaxed scenario where a specific task (τ_5) is permitted to miss its deadline, observing how this impacts the overall optimization matrix. A Branch and Bound algorithmic approach written in C was utilized to ensure a rigorous traversal of the solution space.



SYSTEM MODELING AND WCET EXTRACTION

I.1 WCET Evaluation for τ_1

Task τ_1 involves calculating the product of large randomly generated numbers. To prevent too short execution times due to C compiler optimisation, the number is given as type `versatile long long int`. To acquire an accurate statistical representation of its execution, 1000 compilations and runtime evaluations were simulated.

The empirical tests revealed an execution profile characterized by the following metrics:

- **Q1:** 0.53 ns
- **Median (Q2):** 0.54 ns
- **Q3:** 0.56 ns
- **WCET (C_1):** ≈ 1.3 ns

I.2 Data Scaling Strategy

To implement a precise programmatic scheduler at the job level, computing times strictly as integers is preferred to prevent floating-point inaccuracies and buffer anomalies. Because the observed WCET for τ_1 was 1.3, a scaling factor was universally applied.

All task properties (Computation Time C_i and Period T_i) from the initial dataset were multiplied by a factor of 10. This allows $C_1 = 1.3$ to be safely represented as 13 inside the algorithmic domain without distorting the utilization factor.

Task (T_i)	Computation Time (C_i)	Period (T_i)
τ_1	13	100
τ_2	30	100
τ_3	20	200
τ_4	20	200
τ_5	20	400
τ_6	20	400
τ_7	30	800

Tableau I.1: Scaled Task Set utilized for the Scheduling Algorithm

SCHEDULABILITY AND OPTIMIZATION

II.1 Processor Utilization and Hyperperiod

Before attempting to find a schedule, the theoretical schedulability must be verified. The total processor utilization

U is given by $U = \sum_{i=1}^n \frac{C_i}{T_i}$.

$$U = \frac{13}{100} + \frac{30}{100} + \frac{20}{200} + \frac{20}{200} + \frac{20}{400} + \frac{20}{400} + \frac{30}{800} = 0.7675$$

Since $U = 0.7675 \leq 1$, the task set is theoretically schedulable.

Assumption 1: The Hyperperiod Assumption

To prove permanent schedulability for a periodic task set without simulating infinite time, the schedule only needs to be verified across one **Hyperperiod**. The hyperperiod is the Least Common Multiple (LCM) of all task periods.

$$H = \text{LCM}(100, 100, 200, 200, 400, 400, 800) = 800$$

Over one hyperperiod, exactly 29 distinct jobs are spawned. If a valid, non-overlapping sequence exists for these 29 jobs within $t \in [0, 800]$, the system is indefinitely schedulable.

II.2 Algorithmic Complexity

The scheduler is built on a **Branch and Bound (B&B)** search tree. Let N be the total number of jobs within the hyperperiod ($N = 29$). The problem of sequencing N non-preemptive jobs optimally is NP-Hard.

The worst-case computational complexity is inherently bounded by the permutations of jobs, leading to $\mathcal{O}(N!)$ complexity. However, the B&B algorithm aggressively prunes the search space by bounding:

1. Any branch where the current accumulated wait time exceeds the best-known minimum is instantly pruned.
2. Any branch where the temporal constraints $start_time + C_i > deadline$ are violated is discarded.

II.3 Minimizing Wait Time and Maximizing Idle Time

The instruction set mandates finding a schedule that minimizes total waiting time, which implies that total processor idle time is maximized.

Verification of this relationship:

The absolute sum of execution times over the hyperperiod is constant ($\sum C_i \times \frac{H}{T_i}$). Consequently, the absolute sum of idle time ($H - \text{Active Time}$) is structurally fixed at $800 - 614 = 186$. However, minimizing the waiting delay forces the scheduler to execute jobs immediately upon their release. This consolidation eliminates micro-fragmentation between jobs, thereby grouping the scattered processor idle ticks into the *maximum contiguous idle blocks* possible.

SCHEDULING SCENARIOS AND RESULTS

III.1 Scenario 1: Strict Schedule (No Deadlines Missed)

In the primary scenario, the branch and bound algorithm processes all 29 jobs strictly enforcing deadlines.

- **Optimal Wait Time:** Successfully minimized across the hyperperiod.
- **Constraint Check:** All calculated response times for scheduled jobs strictly remain $R_i \leq D_i$.

By pushing the scheduling routine, the node tree guarantees the optimal path. The resulting schedule and results is given in the following table :

Time Interval	Task Executed	Release Time	Deadline	Wait Time	Response Time
[00 -> 13]	T1	Release: 00	Deadline: 100	Wait: 00	Response: 13
[13 -> 33]	T3	Release: 00	Deadline: 200	Wait: 13	Response: 33
[33 -> 53]	T4	Release: 00	Deadline: 200	Wait: 33	Response: 53
[53 -> 83]	T2	Release: 00	Deadline: 100	Wait: 53	Response: 83
[83 -> 103]	T5	Release: 00	Deadline: 400	Wait: 83	Response: 103
[103 -> 116]	T1	Release: 100	Deadline: 200	Wait: 03	Response: 16
[116 -> 136]	T6	Release: 00	Deadline: 400	Wait: 116	Response: 136
[136 -> 166]	T2	Release: 100	Deadline: 200	Wait: 36	Response: 66
[166 -> 196]	T7	Release: 00	Deadline: 800	Wait: 166	Response: 196
[200 -> 213]	T1	Release: 200	Deadline: 300	Wait: 00	Response: 13
[213 -> 233]	T3	Release: 200	Deadline: 400	Wait: 13	Response: 33
[233 -> 253]	T4	Release: 200	Deadline: 400	Wait: 33	Response: 53
[253 -> 283]	T2	Release: 200	Deadline: 300	Wait: 53	Response: 83
[300 -> 313]	T1	Release: 300	Deadline: 400	Wait: 00	Response: 13
[313 -> 343]	T2	Release: 300	Deadline: 400	Wait: 13	Response: 43
[400 -> 413]	T1	Release: 400	Deadline: 500	Wait: 00	Response: 13
[413 -> 433]	T3	Release: 400	Deadline: 600	Wait: 13	Response: 33
[433 -> 453]	T4	Release: 400	Deadline: 600	Wait: 33	Response: 53
[453 -> 483]	T2	Release: 400	Deadline: 500	Wait: 53	Response: 83
[483 -> 503]	T5	Release: 400	Deadline: 800	Wait: 83	Response: 103
[503 -> 516]	T1	Release: 500	Deadline: 600	Wait: 03	Response: 16
[516 -> 536]	T6	Release: 400	Deadline: 800	Wait: 116	Response: 136
[536 -> 566]	T2	Release: 500	Deadline: 600	Wait: 36	Response: 66
[600 -> 613]	T1	Release: 600	Deadline: 700	Wait: 00	Response: 13
[613 -> 633]	T3	Release: 600	Deadline: 800	Wait: 13	Response: 33
[633 -> 653]	T4	Release: 600	Deadline: 800	Wait: 33	Response: 53
[653 -> 683]	T2	Release: 600	Deadline: 700	Wait: 53	Response: 83
[700 -> 713]	T1	Release: 700	Deadline: 800	Wait: 00	Response: 13
[713 -> 743]	T2	Release: 700	Deadline: 800	Wait: 13	Response: 43

Tableau III.1: Order of job execution and their respective wait and response times for the strict scheduling scenario.

The total wait time of the schedule is 1064

III.2 Scenario 2: Relaxed Constraints (τ_5 Delay)

In the subsequent run, the scheduling strictness is relaxed allowing τ_5 to miss a deadline to potentially decrease the cumulative waiting times of higher-frequency tasks.

When the pruning guardrail for τ_5 's deadline (D_5) is deactivated:

- The algorithm explores previously forbidden permutations where τ_5 yields the processor.
- Because τ_5 has a massive period (400) compared to τ_1 (100), delaying τ_5 past its deadline removes significant wait bottlenecks for the subsequent immediate releases of τ_1 and τ_2 .
- **Result:** Even though τ_5 can miss its deadline, the total wait time for the remaining tasks doesn't move at all. This confirms that the optimal schedule is already achieved under strict constraints.

Time Interval	Task Executed	Release Time	Deadline	Wait Time	Response Time
[00 -> 13]	T1	Release: 00	Deadline: 100	Wait: 00	Response: 13
[13 -> 33]	T3	Release: 00	Deadline: 200	Wait: 13	Response: 33
[33 -> 53]	T4	Release: 00	Deadline: 200	Wait: 33	Response: 53
[53 -> 83]	T2	Release: 00	Deadline: 100	Wait: 53	Response: 83
[83 -> 103]	T5	Release: 00	Deadline: 400	Wait: 83	Response: 103
[103 -> 116]	T1	Release: 100	Deadline: 200	Wait: 03	Response: 16
[116 -> 136]	T6	Release: 00	Deadline: 400	Wait: 116	Response: 136
[136 -> 166]	T2	Release: 100	Deadline: 200	Wait: 36	Response: 66
[166 -> 196]	T7	Release: 00	Deadline: 800	Wait: 166	Response: 196
[200 -> 213]	T1	Release: 200	Deadline: 300	Wait: 00	Response: 13
[213 -> 233]	T3	Release: 200	Deadline: 400	Wait: 13	Response: 33
[233 -> 253]	T4	Release: 200	Deadline: 400	Wait: 33	Response: 53
[253 -> 283]	T2	Release: 200	Deadline: 300	Wait: 53	Response: 83
[300 -> 313]	T1	Release: 300	Deadline: 400	Wait: 00	Response: 13
[313 -> 343]	T2	Release: 300	Deadline: 400	Wait: 13	Response: 43
[400 -> 413]	T1	Release: 400	Deadline: 500	Wait: 00	Response: 13
[413 -> 433]	T3	Release: 400	Deadline: 600	Wait: 13	Response: 33
[433 -> 453]	T4	Release: 400	Deadline: 600	Wait: 33	Response: 53
[453 -> 483]	T2	Release: 400	Deadline: 500	Wait: 53	Response: 83
[483 -> 503]	T5	Release: 400	Deadline: 800	Wait: 83	Response: 103
[503 -> 516]	T1	Release: 500	Deadline: 600	Wait: 03	Response: 16
[516 -> 536]	T6	Release: 400	Deadline: 800	Wait: 116	Response: 136
[536 -> 566]	T2	Release: 500	Deadline: 600	Wait: 36	Response: 66
[600 -> 613]	T1	Release: 600	Deadline: 700	Wait: 00	Response: 13
[613 -> 633]	T3	Release: 600	Deadline: 800	Wait: 13	Response: 33
[633 -> 653]	T4	Release: 600	Deadline: 800	Wait: 33	Response: 53
[653 -> 683]	T2	Release: 600	Deadline: 700	Wait: 53	Response: 83
[700 -> 713]	T1	Release: 700	Deadline: 800	Wait: 00	Response: 13
[713 -> 743]	T2	Release: 700	Deadline: 800	Wait: 13	Response: 43

Tableau III.2: Relaxed schedule ($T5$ allowed to miss deadline)

CONCLUSION

This analysis successfully demonstrates that the provided task set is schedulable under non-preemptive constraints. Through data scaling, floating-point ambiguity was bypassed, enabling a robust Branch and Bound traversal. By mapping out a full hyperperiod ($H = 800$), the scheduler validated that a combination minimizing wait times guarantees maximum contiguous processor idle time. However, allowing τ_5 to miss its deadline did not yield any improvement in the total wait time, confirming that the optimal schedule is already achieved under strict constraints.